

El lenguaje C. El tipo Puntero.

Segunda parte

Profesores Estructuras de Datos, 2026.

Dpto. Lenguajes y Ciencias de la Computación.

University of Málaga

Licenciado bajo [CC BY-NC 4.0](https://creativecommons.org/licenses/by-nc/4.0/)

El tipo Puntero. Por qué es importante

1. Acceso directo a la memoria
2. **Manipulación eficiente de arrays y cadenas de caracteres**
3. Paso por referencia
4. Creación de estructuras de datos complejas
5. Gestión dinámica de la memoria

Punteros y Arrays

- En C, los punteros y los arrays están estrechamente relacionados
- Un array almacena un puntero a su primer elemento. Toda operación sobre un array se puede hacer con punteros
 - **Ventaja:** mayor control.
 - **Inconveniente:** menos legible.

Aritmética de punteros

Aritmética de punteros

- Se permiten comparar direcciones de memoria, un array es un puntero al primer elemento.

```
int a[10];  
int *ptr= a;
```

- Se puede sumar a un puntero. Dependiendo de su tipo, **avanzará** mas o menos bytes en memoria.

```
*(ptr + 1) == a[1]
```

- El operador ++ se puede usar para iterar.

```
ptr++
```

- Podemos **saltar** el puntero i posiciones en el array

```
ptr += i
```



Punteros y arrays. Ejemplo 1

- Declara un array de enteros y un puntero a entero.
- Luego asigna la dirección del primer elemento del array al puntero.
- Finalmente, utiliza un bucle para recorrer el array utilizando el puntero e inicializa sus valores con la posición que ocupa.
- Usa el [depurador](#) para validar su correcto funcionamiento.

Punteros y cadenas de caracteres

- En C, una cadena de caracteres (`string`) es un array de caracteres terminado por el carácter ASCII 0 (`'\0'`)
- Definición de una cadena

```
char cad[5];
```

- **C no controla el acceso a posiciones fuera de la cadena**
- Manejo de la cadena utilizando el nombre del array

```
printf("%s", cad);
```

- Acceso a posiciones individuales, con sintaxis de array

```
printf("%c", cad[0]);
```

Punteros y cadenas de caracteres

- Inicialización
 - En la propia declaración:

```
char cad[5] = "hola";  
char cad[5] = {'h', 'o', 'l', 'a', '\0'};
```

- **No está definido el operador de asignación**

```
cad = "hola";    /* ERROR */
```

- Utilizar strcpy para asignar un valor a una cadena (librería string.h)

```
strcpy(cad, "hola");
```

¿Qué resultado da?

```
//Cadenas sin terminador
#include<stdio.h>

int main(void)
{
    char cad0[4] = {'p', 'e', 'n', 'a'};
    char cad[4] = {'h', 'o', 'l', 'a'};
    char cad1[4] = {'c', 'a', 'r', 'a'};
    printf("%s\n", cad);
    return 0;
}
```

Punteros y cadenas de caracteres

- Algunas funciones útiles de la librería `<string.h>`

```
// Copia src en dest (sin controlar el tamaño de src)
char *strcpy(char *dest, const char *src);
```

```
//Retorna la longitud de str (se supone terminado en \0)
size_t strlen(const char *str);
```

```
//Añade src a dest
char *strcat(char *dest, const char *src);
```

```
//Compara str1 y str2, devuelve:
//0 si son iguales;
//menor que 0 si str1<str2;
//y mayor que 0 si str1>str2
int strcmp(const char *str1, const char *str2);
```



Ejemplo 2

- Define tres cadenas, str1, str2 y str3.
- Inicializa str1 y str2.
- Copia una de esas cadenas en str3
- Concatena str1 y str2
- Compara str1 y str3 y devuelve un mensaje si son diferentes.

Arrays como parámetros en funciones

- Se pasan con sintaxis de punteros

```
void mifuncion(int *param);
```

```
void mostrar(int *elem, int n) {  
    int i;  
    for (i=0; i<n; i++)  
        printf("%d", elem[i]);  
}
```

```
int datos[10];
```

```
....
```

```
//Un array es un puntero a su primer elemento, podemos obviar el &  
mostrar (datos);
```

¿Devolver Arrays en funciones?

- Se pueden devolver punteros (¡cuidado con esto!)
- Ejemplo:

```
char *strcpy(char *destination, const char*source);
```

Arrays y funciones

Si necesitamos pasar un array como parámetro de un función **SIEMPRE** se le pasa la dirección de comienzo de éste

Nunca se pasan por valor (copia de sus elementos)



Punteros y cadenas.

Ejemplo2

- Define tres cadenas, str1, str2 y str3.
- Inicializa str1 y str2.
- Copia una de esas cadenas en str3
- Concatena str1 y str2
- Compara str1 y str3

Documentación: [strcpy](#), [strcat](#), [strcmp](#).



Punteros y arrays. Ejemplo 3

- Declara una función que copie dos cadenas.
- No uses el `strcpy`, prográmalo tú con un bucle y asumiento que la cadena origen tiene el caracter terminador `'\0'`.

Arrays y funciones

Punteros y Registros (Structs)

- Los punteros pueden apuntar a cualquier tipo

Ej:

```
struct Punto{  
    int x,y;  
};
```

```
struct Punto *ppunto;
```

- Con typedef

```
typedef struct Punto *PtrPunto;  
PtrPunto ppunto;
```

Punteros y Registros (structs)

- El acceso a los miembros puede hacerse de dos formas
- Sintaxis 1:

```
(*ppunto).x=20;
```

- Sintaxis 2:

```
ppunto->x=20;
```



Ejemplo 4

- Define una estructura punto con dos enteros, x e y.
- Define un método cambiarPuntos, que intercambia los valores de estos enteros en una estructura.

¿Las estructuras se pasan por valor? ¿es necesario pasar un puntero a ellas para modificarlas?